



ПРАКТИЧЕСКИЙ УРОК

Git: от первого коммита до конфликтов слияния

Единая карта команд, ветвления, отката ошибок
и безопасной синхронизации с удалённым репозиторием



План урока и ожидаемый результат

К концу занятия вы сможете пройти полный цикл и самостоятельно разобрать конфликт.

- Что хранит Git и чем локальный репозиторий отличается от удалённого
- Основной цикл: status → add → commit → push / pull
- История, сравнение, отмена ошибочных действий
- Ветки: создать, переключить, слить; merge и rebase
- Практика в парах: два разработчика меняют один файл, получают и решают конфликт

Маршрут занятия на 90 минут

Можно сократить теорию и оставить больше времени на практику.

0–10 мин

Модель Git и три области

10–30 мин

Базовые команды и история

30–45 мин

Удалённый репозиторий и ветки

45–60 мин

Ошибки, откаты и безопасные команды

60–85 мин

Практика: push, pull, конфликт, решение

85–90 мин

Проверка понимания и выводы

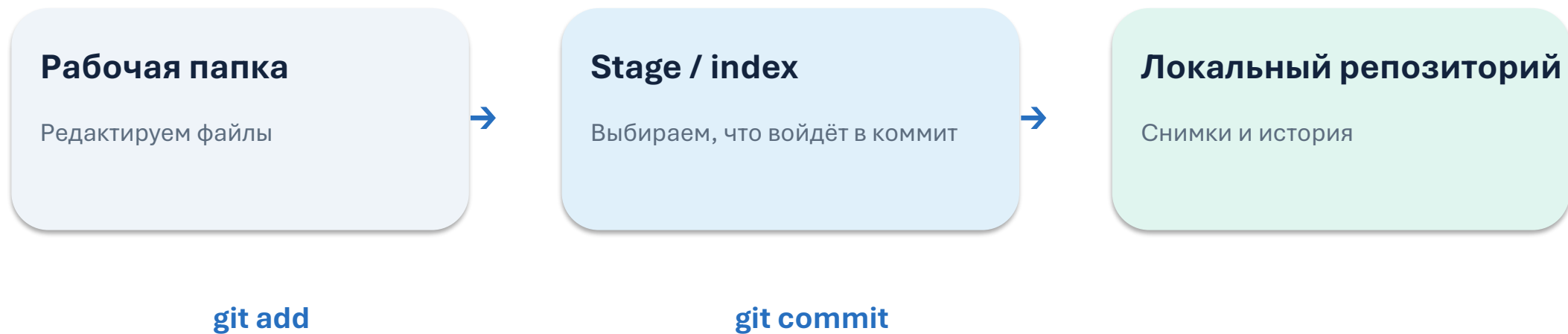
Зачем нужна система контроля версий

VCS хранит историю зафиксированных состояний и позволяет сравнивать и восстанавливать их.

- Совместная работа: несколько участников меняют один проект
- История: кто, когда и зачем внёс изменение
- Эксперименты без риска сломать стабильную версию
- Параллельная поддержка функций и релизов
- Git — распределённая система: полноценная история находится локально

Git: три области и движение изменений

Ключ к пониманию команд — различать рабочую папку, индекс и коммиты.



```
git status # где находятся изменения
```

Установка и первичная настройка

Один раз на компьютере; имя и email попадут в метаданные коммитов.

```
git -version
git config --global user.name "Имя Фамилия"
git config --global user.email name@example.com
git config --global --list
```

- Для удалённого доступа обычно используют SSH или HTTPS
- Настройте доступ к GitHub/GitLab заранее.

Создать репозиторий или получить готовый

Выберите один путь: начать с пустой папки или клонировать проект.

```
git init                # создать репозиторий здесь
git clone <URL>         # скачать репозиторий и историю
cd <папка-проекта>
git status
```

- git init создаёт локальную базу .git
- git clone обычно настраивает удалённый репозиторий origin
- Не запускайте git init внутри уже клонированного проекта без причины

Повседневный цикл: status → add → commit

Коммит — локальная фиксация подготовленного состояния, а не отправка на сервер.

```
git status
git add README.md          # конкретный файл
git add .                  # все изменения в текущем каталоге
git diff                   # изменения вне stage
git diff --staged          # что попадёт в коммит
git commit -m "Add project README"
```

- Проверяйте git diff перед фиксацией
- Добавляйте осмысленные части, а не всё подряд

Полезные команды для файлов и чистоты проекта

Не все файлы должны попадать в историю.

```
git rm file.txt           # удалить и подготовить удаление
git rm --cached file.txt  # убрать из Git, оставить на диске
git mv old.txt new.txt    # переместить/переименовать
git status --short        # .gitignore: build/ *.log .env
```

- Не коммитьте пароли, ключи, токены и локальные настройки
- Если секрет уже попал в историю — простого удаления файла недостаточно

История и сравнение изменений

История отвечает «что и когда», diff — «что именно поменялось».

```
git log --oneline --graph --decorate -all
git show <commit>
git diff <commit-A> <commit-B>
git diff HEAD~1 HEAD
git blame <file>           # строки и авторство
```

- Хэш коммита — идентификатор конкретного состояния
- --graph помогает увидеть ветвления и слияния

Удалённый репозиторий: fetch, pull, push

Локальный commit и удалённая публикация — разные операции.

```
git remote -v
git fetch origin      # получить сведения и объекты, не сливая
git pull              # получить и интегрировать изменения
git push -u origin <branch> # отправить ветку и связать upstream
```

- Перед началом работы: git pull (или fetch + осознанная интеграция)
- Если push отклонён (non-fast-forward), сначала получите чужие коммиты и интегрируйте их
- Не используйте force push в общей ветке без согласования

Ветки: отдельная линия работы

Ветка — лёгкий указатель на коммит; она позволяет вести работу независимо.

```
git branch          # список веток
git switch -c feature/login  # создать и перейти
git switch main      # переключиться
git branch -d feature/login  # удалить слитую ветку
git log --oneline --graph --all
```

- Рабочая схема: main — стабильная; feature/* — отдельная задача; hotfix/* — срочное исправление
- Часто коммитьте в свою ветку; вливайте готовую работу через merge или Pull Request

Слияние: merge и fast-forward

Перейдите в ветку-получатель и выполните merge источника.

```
git switch main  
git merge feature/login  
# если конфликт: исправить файлы → git add → git commit
```

- Fast-forward: указатель main просто передвигается вперёд, если расхождения не было
- Трёхстороннее слияние: Git использует общего предка и две версии
- Merge сохраняет факт интеграции; конфликт не означает потерю данных

Rebase: перепроиграть свои коммиты поверх новой основы

Удобен для локальной feature-ветки; меняет идентификаторы коммитов.

```
git switch feature/login
git fetch origin
git rebase origin/main
# конфликт → исправить → git add <файл>
git rebase -continue
# отменить операцию: git rebase --abort
```

- Merge: история показывает интеграцию веток
- Rebase: более линейная история, но переписываются коммиты
- Не делайте rebase опубликованной общей ветки, если другие уже основаны на её коммитах

Ошибки: безопасно отменяем, не стирая историю

Сначала выясните, где находится изменение: working tree, stage или commit.

```
git restore <file>           # отбросить незастейдженные правки
git restore --staged <file>   # убрать файл из stage
git commit --amend           # исправить последний локальный коммит
git revert <commit>          # новый коммит, отменяющий указанный
git reset --soft HEAD~1      # сдвинуть HEAD, оставить изменения в stage
```

- Опасно: `git reset --hard` и `git clean -fd` могут безвозвратно удалить локальную работу
- Для опубликованных коммитов обычно предпочтительнее `git revert`

Если push отклонён: типовой алгоритм

Причина часто в том, что удалённая ветка содержит коммиты, которых нет локально.

```
git status
git fetch origin
git log --oneline --graph --decorate -all
git pull --rebase origin <branch> # или merge по правилам команды
# разрешить конфликты, продолжить rebase / завершить merge
git push origin <branch>
```

- Не повторяйте push вслепую и не переписывайте общую историю
- Если не уверены — сохраните копию/создайте временную ветку и разберите граф

Конфликт: что Git сообщает и что делать

Конфликт возникает, когда Git не может автоматически объединить пересекающиеся правки.

```
<<<<<< HEAD
текст из текущей ветки
=====
текст из вливаемой ветки
>>>>>> feature/login
```

- Откройте файл и выберите итоговый вариант: одну сторону или осмысленную комбинацию
- Удалите маркеры конфликта, проверьте контекст и тесты
- `git add <файл>` отмечает конфликт решённым; далее `merge commit` или `rebase --continue`

Практика: два разработчика — один файл

Работа в парах. Нужны общий удалённый репозиторий и два клона А и В.

- Подготовьте файл `menu.txt` с двумя строками: «Главная» и «Контакты»
- А и В клонируют репозиторий; каждый создаёт свою ветку или работает в общей учебной ветке по сценарию
- А меняет первую строку на «Главная — А» и коммитит; В меняет ту же строку на «Главная — В» и коммитит
- А отправляет изменения (`push`); В пытается отправить — получает отказ из-за удалённых изменений
- В выполняет `fetch/pull`, получает конфликт; вместе разберите маркеры и выберите итоговую строку

Практика: команды преподавателя и ожидаемый результат

Показывайте команды по мере возникновения ситуации, а не все заранее.

```
# в клоне B
git fetch origin
git pull --no-rebase origin <branch>
git status
# исправить menu.txt, удалить маркеры
git add menu.txt
git commit
git push origin <branch>
```

- Результат: в удалённой ветке есть обе согласованные правки
- Попросите студентов объяснить: почему push отказал, что означает каждый маркер, почему нужен add после исправления

Практика: дополнительные сценарии на выбор

Если основная практика прошла быстро, добавьте одну ошибку на откат.

- Сценарий А — ошибочный коммит ещё локальный: исправить файл и выполнить amend
- Сценарий В — ошибочный коммит уже опубликован: отменить через revert
- Сценарий С — не те файлы попали в stage: `git restore --staged` и повторный add
- Сценарий D — незаконченная работа нужна срочно в другой ветке: `git stash` / `git stash pop`
- Сценарий E — конфликт при rebase: решить, add, rebase --continue или rebase --abort

Стандарт написания коммитов (Conventional Commits)

Формат коммита

Основные типы:

- feat: Новая функциональность (фича).
- fix: Исправление ошибки.
- docs: Изменения в документации.
- style: Форматирование, точки с запятой (без изменения кода).
- refactor: Рефакторинг кода (без исправления ошибок и фич).
- test: Добавление или правка тестов.
- chore: Рутинные задачи, обновление зависимостей.

Основной текст:

Формат сообщения: <тип>: <описание>

Примеры:

- feat: добавлена авторизация через Google
- fix: исправлено падение при загрузке аватара

Мини-шпаргалка: команда → вопрос

Пусть студент выбирает команду по ситуации, а не заучивает список.

Где мои изменения?

`git status`

Что изменилось?

`git diff / diff --staged`

Что войдёт в коммит?

`git add → diff --staged`

Сохранить снимок?

`git commit`

Получить изменения без слияния?

`git fetch`

Получить и интегрировать?

`git pull`

Отправить свои коммиты?

`git push`

Изолировать задачу?

`git switch -c`

Объединить ветки?

`git merge / rebase`

Отменить опубликованный коммит?

`git revert`

Финальная проверка понимания

Попросите ответить без подсказки или показать в терминале.

- Чем `commit` отличается от `push`?
- В чём разница между `fetch` и `pull`?
- Что безопаснее для отмены уже опубликованного коммита — `reset` или `revert`? Почему?
- Что делать после ручного исправления конфликтного файла?
- Когда нельзя делать `rebase`?
- Как понять, какие изменения попадут в следующий коммит?

Итог: безопасный рабочий ритм

Главное правило — проверять состояние до и после каждой значимой операции.

- Часто проверяйте `git status` и изучайте `diff`
- Делайте небольшие осмысленные коммиты
- Работайте в отдельных ветках; интегрируйте регулярно
- Перед `push` синхронизируйтесь с удалённой веткой
- При конфликте не спешите: изучите обе версии, соберите итог, проверьте тестами
- Для опубликованной истории выбирайте `revert`; `destructive`-команды используйте осознанно